

数据结构与算法入门

Category: 计算机 | BookVault

Based on public knowledge. Educational use only.

第一章 算法与复杂度分析

算法是解决特定问题的一组有限、确定、可执行的指令序列。算法的五个基本特性：

有穷性(必须在有限步后终止)、确定性(每一步必须有确切定义)、输入(有零个或多个输入)、输出(至少有一个输出)、可行性(每一步都可以在有限时间内完成)。

算法效率通常用时间复杂度和空间复杂度来衡量。时间复杂度使用大O记号(Big-O

notation)描述执行时间随输入规模 n 增长的趋势上限。常见级别从低到高： $O(1)$ 常数阶， $O(\log n)$ 对数阶， $O(n)$ 线性阶， $O(n \log n)$ 线性对数阶， $O(n^2)$ 平方阶， $O(2^n)$ 指数阶。

例如，在有序数组中二分查找的时间复杂度为 $O(\log$

$n)$ ，因为每次比较将搜索范围减半。冒泡排序需要 $O(n^2)$ 次比较，而归并排序和快速排序(平均情况)仅需 $O(n \log n)$ 。当 $n=10^6$ 时， $O(n \log n)$ 与 $O(n^2)$ 的实际运行时间相差约50000倍。

P与NP问题是理论计算机科学的核心未解难题。P类问题是指可以在多项式时间内求解的问题；NP类问题是指解可以在多项式时间内验证的问题。P=NP?

这一问题被克雷数学研究所列为千禧年七大数学难题之一，悬赏一百万美元。

第二章 基本数据结构

数组(Array)是最基础的数据结构，在内存中连续存储相同类型的元素，支持通过索引 $O(1)$ 时间随机访问。数组的缺点是插入和删除操作需要移动后续元素，时间复杂度为 $O(n)$ 。动态数组通过预留额外空间和倍增策略，将摊销插入时间降至 $O(1)$ 。

链表(Linked

List)由一系列节点组成，每个节点包含数据和指向下一个节点的指针。单链表支持 $O(1)$ 时间在头部插入/删除，但访问第 k 个元素需要 $O(k)$ 时间遍历。双向链表每个节点有两个指针(前驱和后继)，使两端的插入/删除均为 $O(1)$ 。链表不需要连续内存空间。

栈(Stack)是一种后进先出(LIFO)的线性结构，push和pop皆为 $O(1)$ 。栈在函数调用(调用栈)、表达式求值、括号匹配、浏览器的后退功能中都有应用。队列(Queue)是先进先出(FIFO)结构，广泛用于任务调度、消息缓冲和BFS遍历等场景。哈希表通过哈希函数将键映射到桶位置，期望情况下增删查均为 $O(1)$ 。

第三章 树与图

树(Tree)是一种分层结构的非线性数据结构，由 n 个节点和 $n-1$ 条边组成，不存在环路。二叉搜索树(BST)满足左子树所有节点值 $<$ 根节点值 $<$ 右子树所有节点值，平均操作时间复杂度为 $O(\log n)$ ，但最坏情况下(树退化为链表)退化为 $O(n)$ 。

平衡二叉树通过旋转操作维持树的高度为 $O(\log n)$ 。AVL树在每次插入/删除后检查平衡因子并通过旋转恢复平衡。红黑树通过节点着色和较宽松的平衡条件减少旋转次数，在C++

STL的map和Java的TreeMap中被广泛采用。堆(Heap)是一种满足堆序性质的完全二叉树，大顶堆常用于优先队列。

图(Graph)由顶点集 V 和边集 E 组成。图的存储方式主要有邻接矩阵(适合稠密图，空间 $O(|V|^2)$)和邻接表(适合稀疏图，空间 $O(|V|+|E|)$)。深度优先搜索(DFS)使用栈遍历，适合连通性检测和拓扑排序；广度优先搜索(BFS)使用队列，保证找到无权图中的最短路径。